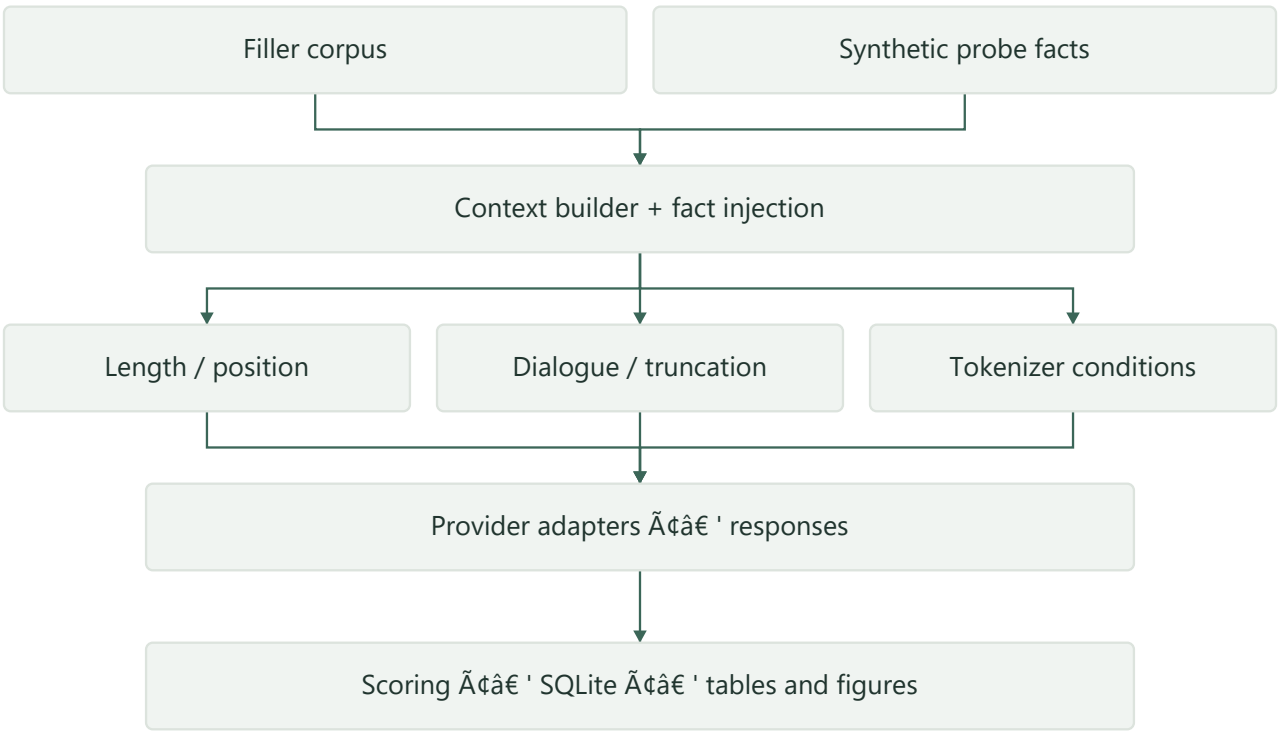
KS-Probe

LLM context-retention benchmarking

An experiment framework that injects controlled facts into long contexts and measures recall across context length, position, dialogue, and truncation conditions.

Architecture



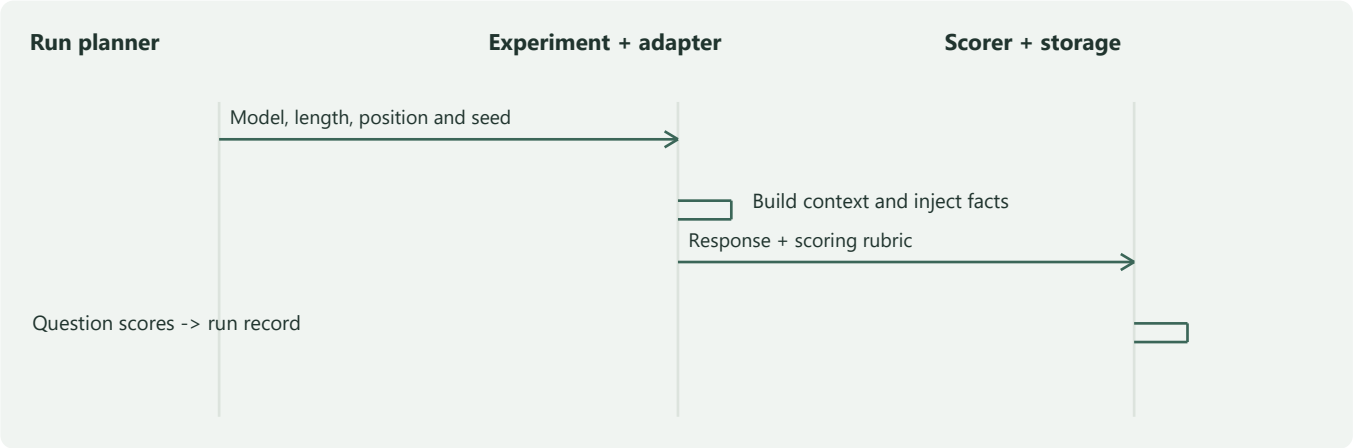
Component overview derived from the supplied source; version-specific details may differ.

Component	Implementation	Role
Corpus + probes	Controlled context construction	Fact placement and questions
Experiments	Five experiment classes	Length, position, turns, truncation, tokens
Adapters	Provider interfaces + mock adapter	Model request abstraction
Orchestrator	Concurrency, rate limits, checkpointing	Repeatable run dispatch
Scoring + records	PRA and SQLite exports	Question and run-level evidence

Benchmark implementation

Experimental unit & run design

KS-Probe treats context retention as a controlled question-answering experiment. A run specification selects a model and condition; the corpus builder provides filler, the injector plants facts and questions, and an adapter returns a response. The question-level score is distinct from the run-level summary and the cross-condition comparison.



Five condition families

Context-length fidelity varies input size; positional recall moves facts within the context; multi-turn degradation tests retention through dialogue; truncation probes inspect what survives shortened input; tokenizer divergence examines tokenization assumptions. These conditions should be analyzed separately before pooling them because their failure mechanisms differ.

Execution and repeatability

The asynchronous orchestrator skips checkpointed specifications and uses concurrency and rate limits. Successful results and failures both matter: silently discarding failed calls changes the evaluation population. Context size should be recorded with the provider/model tokenizer, not only a requested nominal length. Synthetic facts make the ground truth controllable; mock model responses must be labeled separately from provider measurements.

Level	Record needed
Question	Injected gold fact, placement, scoring type and extracted answer
Run	Condition, seed, model identity, request metadata and completion status
Comparison	All planned runs, successful runs, errors and aggregation rule

Source / interpretation: KS_Probe/ks_probe/orchestrator/runner.py; experiments/exp1_context_fidelity.py and other experiment classes; corpus and probe modules.

How Probe Recall Accuracy works

The composite scorer calculates several sub-scores but does not always average them. It selects the result appropriate to the rubric entry, with a hallucination flag taking precedence. This distinction is central to interpreting PRA and to reproducing any benchmark table.

01	Hallucination flagged?	Yes -> PRA = 0
02	Known rubric scoring type?	Exact match / keyword / constraint
03	Unknown type?	Mean of three sub-scores
04	Bound and aggregate	Clip [0,1]; mean across questions

Rule selection

A flagged hallucination sets PRA to zero. Otherwise exact_match produces a binary result, keyword_rule uses its keyword score, and constraint_compliance uses its constraint score. Only an unrecognized scoring type falls back to the average of exact match, keyword and constraint sub-scores. The final result is clipped to [0,1]; mean PRA averages the question-level results and returns zero for an empty list.

Answer extraction is part of the metric

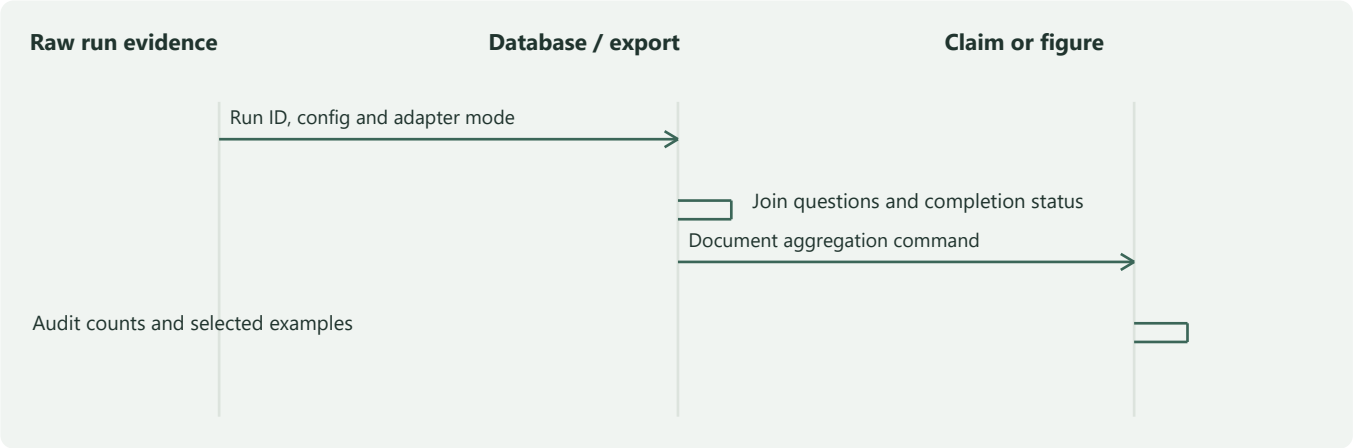
The parser first searches for Q-number delimiters. If it finds none, it assigns the complete response to every question. If only some questions parse, the scoring loop can use the full response for missing ones. This can let unrelated answer text influence a score. A benchmark release should report parser failures and validate a stratified sample against human-readable answers.

Example condition	Scoring effect
Correct text but hallucination flag true	PRA is forced to 0
Known exact-match rubric	Keyword/constraint scores do not replace the binary decision
No Q-number parsing	Whole-response fallback can change question attribution

Source / interpretation: KS_Probe/ks_probe/scorer/composite.py: score_rubric_entry, parse_qa_response, score_all_questions and average_pra.

From run records to claims

An exported CSV or polished figure is the last link in an evidence chain, not its starting point. KS-Probe has useful persistence and reporting infrastructure, but those outputs need to be attached to exact input conditions, adapter modes and raw responses before they support model comparisons.



Provenance constraints

The inspected context-fidelity path stores truncated raw prompt and response strings of up to 2,000 characters. Those records can be helpful for inspection but cannot fully reconstruct every long-context exchange. A separate complete, appropriately handled raw record or content hash and retained artifact is needed for deeper auditing. Reporting scripts also support mock data, so table provenance must identify that mode.

Connection to Kangaroo Shift

Recall from a long context is related to, but different from, effectiveness after context compression or transfer. A proposed paired experiment would compare the same questions after full-context replay, a simple summary baseline and Kangaroo-generated context, while controlling target model and token budget. Retained tasks and decisions should be measured in addition to fact recall. This comparison has not been run as part of the portfolio preparation.

Claim	Minimum supporting evidence
Model ranking	Matched conditions, repeat counts, errors and verified live adapter
Compression benefit	Paired transfer experiment with baseline and token budget
Reproducibility	Code/config hashes, complete response provenance and scorer version

Source / interpretation: KS-Probe experiment persistence and reporting scripts; local logs and tables. Existing aggregate results are not endorsed as reconciled live-provider measurements.

Evidence & source

Implementation, supporting records, and the boundary of the claim

Area	Evidence	Reference
Experimental scope	Five condition families	README and experiment classes
Recorded artifacts	Database, logs, CSVs, figures	Local benchmark folder
Result status	Provider rankings not yet endorsed	Mock/live and run provenance require reconciliation

Selected implementation files

File / record	Path / reference
runner.py	KS_Probe/ks_probe/orchestrator/runner.py
composite.py	KS_Probe/ks_probe/scorer/composite.py
exp1_context_fidelity.py	KS_Probe/ks_probe/experiments/exp1_context_fidelity.py

Selected orchestration, scoring, and experiment modules. API keys and raw database contents are excluded.

Evidence boundary

Synthetic facts are controlled benchmark inputs. Generated mock results must remain distinct from real provider measurements. Existing exported tables are not presented as verified model rankings here.

Technical note

The composite scorer combines exact match, keyword rules, constraint compliance, and a hallucination flag. Answer parsing and truncated stored raw responses are evaluation assumptions that need checking.